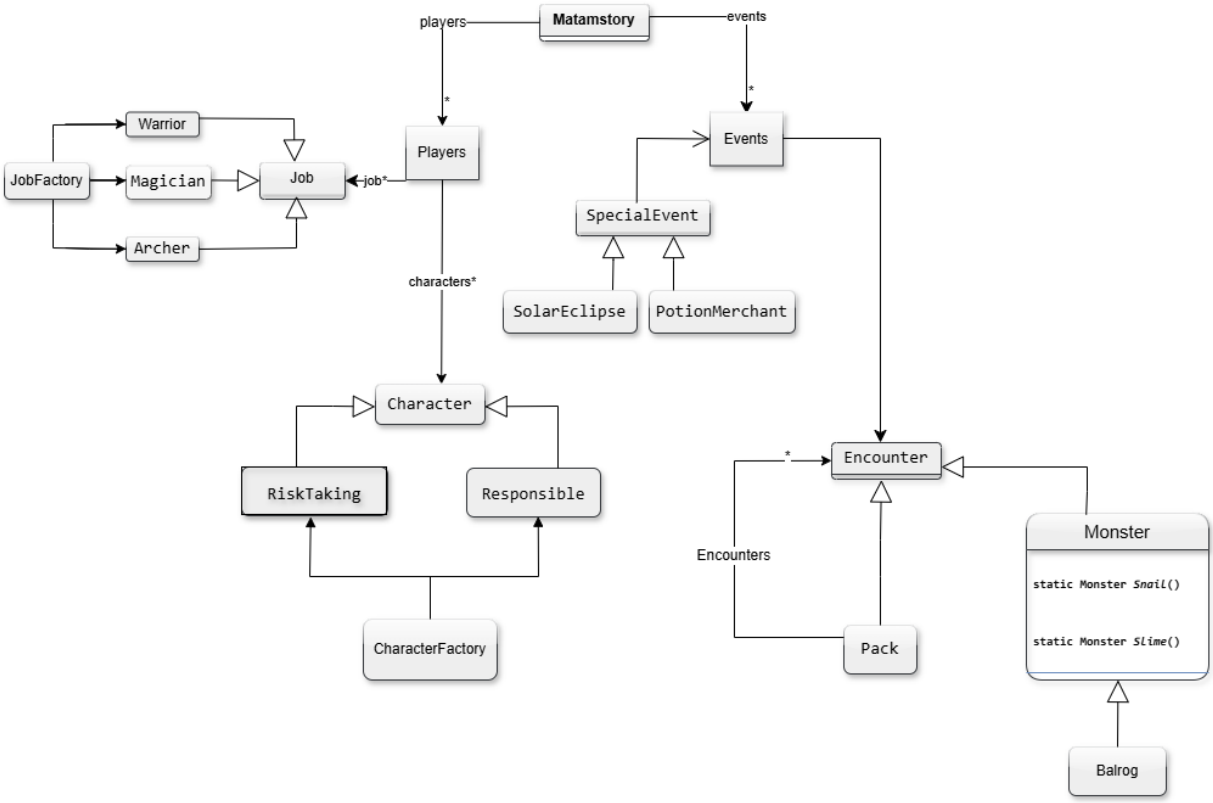
Matam HW4 dry

1. Question 1:UML:



UML: we didn't create classes for snail and slime since they have static values. but rather, we contained them as static functions in the class Monster as appears in the diagram

2.Quastion 2: Design Patterns used:

1) Composite:

As shown on the right side of the UML diagram, the Pack class (which inherits from Encounter) contains multiple encounters. This follows the Composite Design Pattern, where a class that derives from Encounter is an encounter itself while also having encounters within it.

2) Strategy:

We also used the Strategy Design Pattern in the classes *Job* and *Character* in order to define different behaviors based on a player's job and character type. Instead of embedding specific behaviors within the Player class, we created the given job types (Warrior, Magician, Archer) and character traits (RiskTaking, Responsible) as separate

classes. This allows us to dynamically assign unique behaviors to each player, making the system more flexible and extensible.

3) Command:
we used the Command Design Pattern in the Event class hierarchy. We first divided events into two categories: SpecialEvent and Encounter, then we repeated this same prosses in each derived class: Encounter is further divided into Monster and Pack, while Monster has its own derived class, Balrog. This structure allows each event to encapsulate its own behavior, making it easy to introduce new event types without modifying the existing code.

4) Factory:
We implemented the Abstract Factory Pattern for the creation of *Job* and *Character* instances, ensuring flexible approach to object instantiation. This was achieved using a map of *shared pointers functions*, allowing dynamic creation of different job and character types without the need for direct instantiation. The JobFactory and CharacterFactory maintain maps that associate string keys with functions that return shared pointers to newly created objects. This design centralizes object creation, making it easier to extend and manage new jobs and character types .

Question 3:

To add the *Rogue* job to our implementation, we need to create a “Rogue” class as a subclass of *Job* class, overriding the *fight* method to incorporate the escaping mechanic. To support this, several but also minor modifications are required to the code. The class *JobFactory* must be updated to recognize "Rogue" and create an instance accordingly by adding it to the map field in JobFactory class, which serves as the factory for our *Job* class. Additionally, the *handleEvent* function in the *Player* class needs to be modified so that when a “Rogue” encounters an event, the game checks whether they should escape instead of taking damage. The *Encounter* class also requires an update in *applyEvent* method to ensure the Rogue’s escape mechanic is correctly processed. These changes will fully integrate the “Rogue” job into our game with its unique escape ability.

Question 4:

Implementing the Rogue job and the Divine Inspiration event requires only minor changes in the code due to our good design.

In addition to the changes made in the previous question, we have to implement $SpecialEvent$ derived class named $DivineInspiration$, and overriding the *applyEvent* method in it, then as in the same way we did with *Potion merchant event* and *solar eclipse event* we have to implement $divineInspiration$ private method in $Player$, which changes the job pointer to the new (random) job. And finally, we would make

$DivineInspiration$ a friend of $Player$, allowing it call the player method $divineInspiration$ in *handleEvent* (this is a method in class *player* that works with the same functionality of *applyEvent*).

These modifications integrate seamlessly into the existing system thanks to the current design and This modular architecture of the code